

PROJECT PROPOSAL

Parallelizing All-Pairs Shortest Path for Fast City-Scale GPS Route Computation

Using OpenMP (Shared-Memory Parallelism)

Student Name	Huzaifa Arshad
Registration No.	2023-CS-86
Course	Parallel and Distributed Computing (Lab)
Target Environment	OpenMP, C++
Project Type	Software-based

1. Real-World Problem

Navigation and routing systems such as Google Maps, Uber, and parcel-delivery platforms must continuously answer one essential question: what is the fastest way to travel from one location to another across a city? A city map can be represented as a graph, where road intersections act as nodes, roads act as edges, and each edge carries a weight representing distance or travel time.

To respond to user requests instantly, routing backends frequently pre-compute the shortest travel cost between every pair of intersections in advance. This way, when a route is requested, the answer is retrieved immediately rather than calculated from scratch. This is known as the All-Pairs Shortest Path (APSP) problem.

The challenge is that APSP is computationally expensive. The widely used Floyd-Warshall algorithm solves it in $O(N^3)$ time, where N is the number of intersections. For a realistic city containing thousands of intersections, this becomes slow when executed on a single processor core. This makes it an ideal and genuinely useful candidate for parallelization. The project is entirely software-based, requiring only a C++ compiler and the OpenMP library — no specialized hardware is involved.

2. Sequential Approach

The sequential solution implements the standard Floyd-Warshall algorithm. The road network is stored as a two-dimensional distance matrix, where each entry holds the current best-known travel cost between a pair of intersections. Entries with no direct connection are initialized to infinity, and the cost from any intersection to itself is set to zero.

The algorithm then proceeds by considering every intersection in turn as a possible intermediate stop. For each candidate intermediate point, it examines every pair of intersections and checks whether travelling through that intermediate point produces a shorter route than the best route found so far. If it does, the matrix is updated with the improved cost.

This results in three nested loops — one over the intermediate point and two over every source-destination pair — which together produce the algorithm's $O(N^3)$ running time. In testing, a sequential implementation processing 1,000 intersections completed in approximately 1.2 seconds on a single core, with the runtime growing cubically as the number of intersections increases. This rapid growth is precisely the performance bottleneck the parallel version is designed to address.

3. Parallel Implementation Description

The key insight enabling parallelization is the data independence within each stage of the algorithm. When a particular intermediate intersection is being considered, the update applied to each row of the distance matrix depends only on that fixed intermediate point — not on the other rows. As a result, all rows can be updated simultaneously without interfering with one another.

This leads to a clean and reliable parallelization strategy. The outer stage remains sequential, because each stage builds upon the results of the previous one, so the order of intermediate points must be preserved. The row updates, however, are performed in parallel: the available CPU threads each take a portion of the matrix rows and update them independently and concurrently. Because only the row-processing work is distributed across threads, the parallel logic remains almost identical to the sequential version and is easy to validate.

Correctness will be confirmed by comparing the parallel output directly against the sequential output. Because both must produce exactly the same distance matrix, any mismatch would reveal an error. In testing, both versions produced identical results, confirming that the parallelization introduces no race conditions or inconsistencies.

4. Target Environment

The project targets OpenMP, a shared-memory parallel programming model, implemented in C++. OpenMP is the most appropriate choice for this problem for several reasons. The workload consists of tight numerical loops operating on a shared matrix, which is exactly the type of computation OpenMP is designed to accelerate. All threads can access the same distance matrix directly in memory, eliminating any need to copy data between separate processes. Finally, it requires only a single compiler flag to enable and runs on any standard multi-core laptop, with no cluster or distributed setup required.

The same algorithm can later be scaled to a distributed cluster using MPI (Message Passing Interface), in which the matrix rows are partitioned across multiple machines and the relevant data is shared between them at each stage. This is identified as future work to demonstrate the project's potential to scale beyond a single computer.

5. Expected Outcomes: Speedup and Execution

Because the overwhelming majority of the algorithm's work lies in the parallelizable row-update stage, the solution is expected to scale well as more CPU cores are used. According to Amdahl's Law, the achievable speedup is limited mainly by the small

sequential portion of the algorithm and by memory bandwidth. The table below presents the projected performance for a network of 1,000 intersections:

Cores / Threads	Expected Time (N = 1000)	Expected Speedup
1 (sequential)	~1.20 s	1.0× (baseline)
2	~0.65 s	~1.8×
4	~0.35 s	~3.4×
8	~0.20 s	~6.0×

Note: the figures above are projected estimates for a typical 4–8 core laptop. The actual measured values will be recorded and reported in the final submission.

Measurement Plan

The performance of the parallel implementation will be evaluated systematically as follows:

- Run both the sequential and parallel versions on identical input sizes (for example, 500, 1000, and 2000 intersections) and record the wall-clock execution time for each.
- Calculate the speedup as the sequential execution time divided by the parallel execution time, and the efficiency as the speedup divided by the number of threads used.
- Plot speedup against the number of threads to illustrate how well the solution scales and to identify the point at which Amdahl's Law begins to limit further improvement.
- Confirm correctness throughout by verifying that the parallel result matches the sequential result exactly.

6. Related Work

Parallelizing the Floyd-Warshall algorithm for the all-pairs shortest path problem is a well-studied area, and several researchers have demonstrated that significant speedups are achievable. This provides strong support for the approach taken in this project.

Jaiswal and Singh [1] parallelized both the standard and tiled versions of Floyd-Warshall using OpenMP, distributing the rows of the intermediate matrix across threads with the parallel-for directive — the same core strategy adopted in this proposal. Katuka [2] presented serial, regular parallel, and blocked parallel implementations and reported a speedup of roughly 1.5 times for the regular parallel version and up to 4.9 times for the cache-optimized blocked version on an 8-thread Intel processor, confirming the kind of

speedup expected here. More recently, Calderón, Rucci, and Chichizola [3] proposed an enhanced OpenMP algorithm for x86 processors that further improved performance by up to 23 percent through better concurrency. Finally, Jindal et al. [4] applied parallel Floyd-Warshall and Dijkstra algorithms specifically to large-scale transportation networks — the exact real-world setting of this project — and found that combining MPI with shared-memory threads such as OpenMP is highly effective and scalable for city-scale routing.

Together, these works confirm that the chosen problem, algorithm, and OpenMP-based parallelization strategy are both established and practical, while leaving room to measure and analyze the speedup on a standard multi-core machine.

7. Summary

This project addresses a practical and widely used problem — computing the fastest routes across a city road network — and accelerates its $O(N^3)$ Floyd-Warshall solution using OpenMP. The sequential and parallel versions remain closely aligned, correctness is straightforward to verify, and the anticipated multi-core speedup provides a clear and measurable result. The proposal therefore satisfies every requirement of the lab: a genuine real-world problem, a defined sequential baseline, a described parallel implementation, a justified target environment (OpenMP), and concrete expected outcomes for both speedup and execution.

References

- [1] G. Jaiswal, “Parallel Implementation and Analysis of All Pairs Shortest Path Algorithm (Floyd-Warshall Algorithm),” Indian Academy of Sciences Summer Research Report. Available: <http://reports.ias.ac.in/report/12778/>
- [2] J. Katuka et al., “Towards Performance Improvement of a Parallel Floyd-Warshall Algorithm Using OpenMP and Intel TBB,” ResearchGate, 2021. Available: <https://www.researchgate.net/publication/350236410>
- [3] S. Calderón, E. Rucci, and F. Chichizola, “Enhanced OpenMP Algorithm to Compute All-Pairs Shortest Path on x86 Architectures,” arXiv:2403.18619, 2024. Available: <https://arxiv.org/abs/2403.18619>
- [4] V. Jindal et al., “Finding All-Pairs Shortest Path for a Large-Scale Transportation Network Using Parallel Floyd-Warshall and Parallel Dijkstra Algorithms,” Journal of Computing in Civil Engineering. Available: <https://www.researchgate.net/publication/277684075>